

Relazione C++ Tree

Nome: Davide
Cognome: Molteni
Matricola: 909812
E-Mail: d.molteni17@campus.unimib.it

1. Introduzione

Il progetto è costituito dall'implementazione di una classe generica che implementa un albero di ricerca binario di elementi di tipo generico T.

In questo documento vengono descritte le scelte progettuali di implementazione del codice.

2. Implementazione dell'Albero Binario di Ricerca

Per immagazzinare i dati dell'albero di ricerca binario si utilizza una struttura interna privata di nodi: ogni nodo possiede al proprio interno il dato T, un puntatore al nodo destro, un puntatore al nodo sinistro e un terzo puntatore al nodo parente (se presente).

Tra i requisiti del progetto c'era anche la possibilità di sapere sempre il numero di elementi inseriti all'interno dell'albero: questo requisito è soddisfatto dall'utilizzo di una variabile privata `numberOfElements` e dal metodo pubblico per ottenerla `size()`.

2.1 Struttura dati

L'albero è rappresentato da una serie di nodi concatenati in successione; l'albero possiede un attributo `root`, il quale rappresenta un puntatore alla radice dell'albero e dalla quale è possibile percorrere la sua interezza.

Come spiegato in precedenza, ogni nodo possiede quattro attributi:

- `T data` : rappresenta il dato immagazzinato dal `Node` ;
- `Node* left` : rappresenta il nodo sinistro;
- `Node* right` : rappresenta il nodo destro;
- `Node* parent` : rappresenta il nodo padre.

L'aggiunta del nodo padre è necessaria per il traversamento *inorder* dell'albero, la cui implementazione verrà spiegata in futuro.

2.2 Ordinamento e comparatore

Uno dei requisiti del progetto consiste nella possibilità per l'utente di scegliere liberamente la strategia utilizzata per confrontare due elementi di tipo `T`. Per soddisfare questo requisito, la classe `Tree` è parametrizzata tramite un secondo parametro template `Compare`:

```
template<typename T, typename Compare = std::less<T> >
class Tree;
```

Il parametro `Compare` rappresenta un oggetto funzione utilizzato per stabilire l'ordinamento degli elementi all'interno dell'albero. Se l'utente non specifica alcun comparatore, l'opzione di default è utilizzare `std::less<T>` che implementa il confronto tramite l'operatore `<`.

Tutte le operazioni che richiedono confronti tra elementi, come l'inserimento, la ricerca e persino la verifica dell'uguaglianza, utilizzano il comparatore fornito dall'utente.

Poiché il comparatore definisce solamente una relazione di ordinamento e non di controllo di uguaglianza, è stato implementato il metodo privato `isEqual`: due elementi vengono considerati uguali solamente se nessuno dei due risulta minore dell'altro. Questa soluzione consente di determinare l'equivalenza tra due elementi senza richiedere l'implementazione dell'operatore `==` per il tipo `T`.

3. Classe Tree

La classe `Tree` è basata sulla struttura dati appena descritta ed implementa un albero binario di ricerca: la caratteristica principale della struttura dati è che ogni elemento minore dell'elemento corrente venga inserito nel sottoalbero sinistro, e che ogni elemento maggiore dell'elemento corrente venga inserito nel sottoalbero destro. Per questo i confronti sono particolarmente importanti.

La classe `Tree` possiede tre attributi:

- `private Node* root`: puntatore alla radice dell'albero, il punto di entrata;
- `private int numberOfElements`: contatore interno che viene modificato ogni volta che si aggiunge o rimuove un elemento all'albero;
- `private Compare compare`: funtore con il quale si può comparare due tipi `T`.

3.2 Costruttori

La classe `Tree` espone i seguenti costruttori:

3.2.1 Costruttore di default

```
Tree();
```

Inizializza l'albero vuoto, ponendo `root` a `nullptr` e `numberOfElements` a zero.

3.2.2 Copy constructor

```
Tree(const Tree& tree);
```

Inizializza un albero vuoto e lo riempie con una copia di tutti i nodi dell'albero specificato: ciò è fatto tramite la funzione interna ricorsiva `recursiveInsert(const Node* node)`.

3.2.3 Costruttore da intervallo (range constructor)

```
template<typename Iterator>  
Tree(Iterator begin, Iterator end);
```

Inizializza un albero vuoto e lo riempie con una copia dei valori situati nell'intervallo definito dagli iteratori `begin` e `end`.

Il costruttore utilizza iteratori generici, come richiesto dalle specifiche del progetto. Questa scelta rende l'interfaccia indipendente dal contenitore utilizzato e permette di costruire un albero a partire da qualsiasi struttura dati che esponga una coppia di iteratori compatibili. Durante l'inserimento degli elementi, inoltre, viene utilizzato il metodo `insert`, garantendo così che non possano essere introdotti elementi duplicati. Qualora venga rilevato un duplicato, il costruttore svuota l'albero e lancia un'eccezione custom di tipo `DuplicateElementException`.

3.3 Metodi privati

La classe `Tree` possiede alcuni metodi privati, utilizzati come appoggio per evitare la duplicazione di codice.

3.3.1 isEqual

```
bool isEqual(const T& firstElement, const T& secondElement) const;
```

Metodo interno utilizzato per verificare l'eguaglianza tra due dati di tipo `T`. Come spiegato in precedenza, consideriamo due dati di tipo `T` uguali se nessuno dei due è considerato minore dell'altro; siamo in grado di controllare questa proprietà grazie al funtore `Compare`.

3.3.2 find

```
const Node* find(const T& data) const;
```

Metodo interno utilizzato per trovare un determinato dato di tipo `T` all'interno dell'albero. Restituisce un puntatore al nodo contenente il dato specificato oppure `nullptr` se il dato non è presente nell'albero.

3.3.3 clear

```
void clear(Node* node);
```

Metodo interno utilizzato per svuotare e deallocare tutti i nodi dell'albero. Il metodo utilizza una visita post-order, deallocando prima i sottoalberi sinistri, i sottoalberi destri e infine il nodo corrente. Questo garantisce che nessun nodo venga eliminato prima dei suoi figli.

3.3.4 recursiveInsert

```
void recursiveInsert(const Node* node);
```

Metodo interno utilizzato per popolare un albero con i dati contenuti in un altro sottoalbero (la cui radice è appunto `node`).

Il metodo viene utilizzato dal copy constructor e dal metodo `subtree` per costruire nuovi alberi a partire da sottoalberi esistenti.

L'inserimento avviene tramite il metodo pubblico `insert` , evitando di duplicare logica di inserimento e garantendo il mantenimento delle proprietà dell'albero.

4. Metodi Pubblici

La classe `Tree` mette a disposizione diversi metodi pubblici:

4.1 Operatore di assegnazione

```
Tree& operator=(const Tree& other);
```

Sostituisce il contenuto dell'albero corrente con una copia dell'albero specificato. Il contenuto dell'albero viene prima eliminato tramite una chiamata a `clear()` e successivamente viene ricostruito utilizzando `recursiveInsert` sulla radice dell'albero specificato.

Il metodo ritorna un riferimento all'albero corrente.

4.2 size

```
int size() const;
```

Restituisce il numero di elementi contenuti dall'albero.

Il metodo esiste per mantenere la proprietà `numberOfElements` privata, agendo dunque da getter sicuro.

4.3 clear

```
void clear();
```

Dealloca tutti i nodi contenuti nell'albero e lo riporta alle condizioni iniziali.

Fa ciò tramite a una chiamata al metodo privato `clear`, passando come argomento la radice dell'albero corrente.

L'implementazione senza alcun parametro è stata scelta per mantenere privata la classe `Node` all'interno dell'albero.

4.4 contains

```
bool contains(const T& data) const;
```

Verifica che un determinato dato di tipo `T` sia presente all'interno dell'albero.

La ricerca sfrutta le proprietà dell'albero binario di ricerca (valori minori a sinistra, valori maggiori a destra), evitando di visitare nodi non necessari.

4.5 insert

```
void insert(const T& data);
```

Inserisce un dato di tipo `T` all'interno dell'albero.

Internamente, si occupa della creazione di un nuovo nodo e del suo corretto inserimento all'interno dell'albero, mantenendo le sue proprietà.

Può tirare un'eccezione custom `DuplicateElementException` nel caso in cui l'utente provi ad inserire un elemento già presente all'interno dell'albero.

4.6 subtree

```
Tree subtree(const T& data) const;
```

Crea un nuovo albero partendo dal nodo contenente il valore specificato; fa ciò utilizzando il metodo interno `recursiveInsert` e trova il nodo specificato utilizzando il metodo interno

`find` .

Se il valore specificato non è presente, restituisce un nuovo albero vuoto.

5. Iteratore

Come richiesto dal progetto, la classe `Tree` definisce un tipo `const_iterator` che rappresenta un iteratore di sola lettura.

L'iteratore è implementato come classe interna alla classe `Tree` e consente l'attraversamento dell'albero binario di ricerca.

La classe `Tree` espone inoltre i metodi `begin` ed `end` per consentire l'utilizzo dell'iteratore.

5.1 Interfaccia

L'iteratore supporta le operazioni fondamentali, tra cui:

- Dereferenziazione (`operator*`);
- Avanzamento (`operator++`);
- Confronto (`operator!=`).

5.1.1 `operator*`

```
const T& operator*() const;
```

L'operatore di dereferenziazione ritorna il dato di tipo `T` contenuto all'interno del nodo corrente.

5.1.2 `operator++`

```
const_iterator& operator++();
```

L'operatore di pre-incremento avanza l'iteratore all'elemento successivo.

L'attraversamento dell'albero è effettuato tramite una visita *inorder*: vengono visitati prima il sottoalbero sinistro, successivamente il nodo corrente e infine il sottoalbero destro. Con una visita *inorder* gli elementi vengono restituiti secondo l'ordinamento definito dal comparatore (dalla funzione `compare`), restituendo gli elementi dal più piccolo al più grande.

Il solo motivo per cui la classe `Node` debba avere il puntatore al padre è la visita *inorder*: senza di esso, infatti, non riusciremmo a risalire l'albero.

5.1.3 `operator!=`

```
bool operator!=(const const_iterator& other) const;
```

L'operatore di disuguaglianza confronta due iteratori verificando se puntano a nodi differenti. Viene utilizzato principalmente durante l'attraversamento dell'albero per verificare se è stato raggiunto l'operatore di fine (`end()`).

5.2 begin ed end

La classe `Tree` espone i metodi `begin()` ed `end()` per consentire l'utilizzo dell'iteratore.

```
const_iterator begin() const;
```

Il metodo `begin()` restituisce un iteratore al primo elemento della visita inorder. Per ottenere tale elemento, il metodo parte dalla radice dell'albero e raggiunge il nodo più a sinistra.

```
const_iterator end() const;
```

Il metodo `end()` restituisce invece un iteratore che rappresenta la fine della sequenza. Internamente, è rappresentato da un iteratore il cui puntatore al nodo corrente è `nullptr`.

5.3 Iteratore a sola lettura

Come richiesto dalle specifiche del progetto, l'iteratore implementato è un iteratore di sola lettura.

Tale proprietà è garantita dalla firma dell'operatore di dereferenziazione:

```
const T& operator*() const;
```

6. Funzioni Globali

6.1 Operatore di inserzione (operator<<)

```
template<typename T, typename Compare>  
std::ostream& operator<<(std::ostream& os, const Tree<T, Compare>& tree);
```

L'operatore di inserzione consente di stampare il contenuto dell'albero su uno stream di output. L'implementazione sfrutta l'iteratore definito nella classe `Tree`: gli elementi dell'albero compresi tra `begin()` ed `end()` vengono visitati e stampati nella vista specificata prima (dal minore al

maggiore).

L'operatore è implementato come funzione esterna e non come metodo della classe `Tree` in quanto l'operando sinistro è un oggetto di tipo `std::ostream` e non un oggetto di tipo `Tree`. Al termine dell'operazione viene restituito lo stream risultante, consentendo la concatenazione di più operazioni di output.

E' richiesto che per il tipo `T` sia presente un overload dell'operatore `<<`.

6.2 printIF

```
template<typename T, typename Compare, typename P>
void printIF(const Tree<T, Compare>& tree, const P& predicate);
```

Come da requisiti, la funzione `printIF` è globale.

E' utilizzata per stampare solamente gli elementi dell'albero che soddisfano un determinato predicato.

Similarmente a come avviene per l'operatore di inserzione, la funzione attraversa l'intero albero utilizzando l'iteratore di sola lettura. Per ogni elemento viene invocato il predicato `predicate`: se il risultato della valutazione è `True`, l'elemento viene stampato a schermo.

7. Gestione delle Eccezioni

7.1 DuplicateElementException

Per soddisfare il requisito che vieta la presenza di elementi duplicati all'interno dell'albero, è stata definita l'eccezione custom `DuplicateElementException`, derivata dalla classe standard `std::logic_error`.

L'eccezione viene sollevata dal metodo `insert` quando si tenta di inserire un elemento già presente nell'albero. Il controllo viene effettuato tramite il metodo `isEqual`.

7.2 Gestione dei duplicati nel costruttore da intervallo

Il costruttore da intervallo costruisce l'albero inserendo uno alla volta tutti gli elementi compresi nell'intervallo definito dagli iteratori `begin` ed `end`.

Poiché gli elementi vengono inseriti tramite il metodo `insert`, anche durante questa fase può verificarsi il tentativo di inserire un dato duplicato. In tale situazione il metodo `insert` solleva una `DuplicateElementException`.

Per evitare che l'oggetto rimanga in uno stato parzialmente costruito, il costruttore intercetta l'eccezione, svuota completamente l'albero tramite il metodo `clear()` e infine rilancia l'eccezione al chiamante.

In questo modo si garantisce che, in caso di errore, nessuna memoria allocata durante la costruzione dell'albero venga persa e che l'oggetto non contenga una struttura dati incompleta.

8. Gestione della Memoria

8.1 Allocazione dei nodi

L'albero utilizza l'allocazione dinamica della memoria per la creazione dei nodi. Ogni nuovo elemento inserito nell'albero viene memorizzato all'interno di un oggetto `Node`, allocato tramite l'operatore `new`.

L'allocazione avviene solamente all'interno del metodo `insert`, che rappresenta l'unico punto dell'implementazione in cui vengono creati nuovi nodi, evitando la duplicazione di codice.

8.2 Deallocazione della memoria

La deallocazione della memoria è affidata al metodo privato ricorsivo `clear(Node* node)`, utilizzato per liberare tutti i nodi appartenenti ad un sottoalbero.

Il metodo viene utilizzato sia dal metodo pubblico `clear()` che dal distruttore della classe, assicurando il corretto rilascio di tutta la memoria allocata dinamicamente durante il ciclo di vita dell'albero.

9. Conclusioni

9.1 Test eseguiti

Per verificare il corretto funzionamento della libreria, sono stati creati diversi test automatici all'interno del file `main.cpp`.

La lista dei test è la seguente:

```
testAlberoVuotoInt();
testSingoloElementoInt();
testInserimentiMultipliInt();
testInserimentoDuplicatoInt();
testCostruttoreDaIntervalloInt();
testEccezioneCostruttoreDaIntervalloInt();
testCopyConstructorInt();
testSubtreeInt();
testIteratorInt();
```

```
testClearInt();
testPrintIfInt();
testAssignmentOperatorInt();

testCostruttoreDaIntervalloPerson();
testInserimentoDuplicatoPerson();
testSubtreePerson();
testPrintIfPerson();
```

I test sono stati eseguiti sia utilizzando tipi primitivi (i test che finiscono in `Int`) sia utilizzando un tipo personalizzato (i test che finiscono in `Person`) con funtore di comparazione dedicato.

9.2 Valgrind

Per verificare la corretta gestione della memoria dinamica, è stato utilizzato il tool *Valgrind* che ha confermato l'assenza di memory leak e di accessi non validi durante l'esecuzione dei test:

```
==14956== HEAP SUMMARY:
==14956==      in use at exit: 0 bytes in 0 blocks
==14956==    total heap usage: 198 allocs, 198 frees, 81,132 bytes allocated
==14956==
==14956== All heap blocks were freed -- no leaks are possible
==14956==
==14956== For lists of detected and suppressed errors, rerun with: -s
==14956== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

9.3 Doxygen

Il codice è stato documentato tramite *Doxygen*, fornendo descrizioni per classi e metodi.